One timeline, several different questions

Scheduling chooses who runs next. Fairness, throughput, response and deadlines then ask different questions about that choice. **A smaller average waiting time can coexist with a later completion for one job.**

Progression	Question to reconstruct
States and events	Waiting for CPU or I/O? Does a new arrival actually preempt?
General policies	What do FCFS, SJF, SRTF, RR and feedback queues select now?
Multicore and deadlines	How do migration costs and timing constraints change the result?
System cases	Which version, timer and weight rules produce this schedule?

Prerequisites: process/thread states, context switching and basic algebra. Every numerical example fixes its units and assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ready, running and blocked

Ready means able to proceed except for CPU access; **running** means currently executing; **blocked** means awaiting I/O or an event that CPU time alone cannot supply.

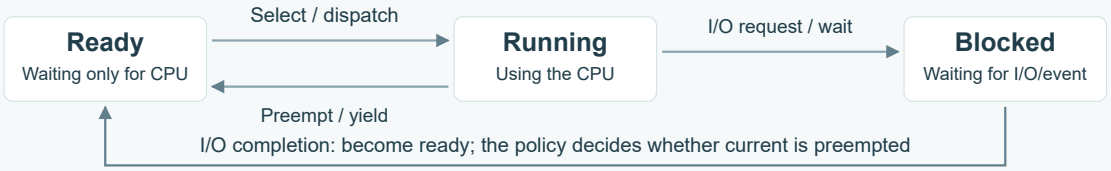
A CPU burst is the CPU demand before the next blocking event or completion. Preemption pauses that demand; it neither finishes nor restarts it. The short-term scheduler selects an eligible execution context; the dispatcher saves/restores context and transfers control.

The ready queue is a candidate set, not necessarily a FIFO implementation: it may use priority queues or a tree. One modeled CPU runs at most one task, and a task cannot be both ready and running.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which events require a new task, and which only change the candidates?

A state change and a dispatch decision are separate steps



New arrivals enter ready; completed tasks leave the system (exit omitted).

Blocked tasks are ineligible; preempted tasks retain their remaining CPU demand.

Blocking or completion requires selecting another ready job or idling. Arrival or wakeup only adds a candidate; **the policy decides whether to preempt**.

Nonpreemptive policies wait for completion, blocking or yield; preemptive policies may withdraw CPU access sooner.

Busy time need not be useful application work

One CPU executes P1 for 5 ms before it blocks, switches for 1 ms, then executes P2 for 2 ms.

Interval (ms)	CPU activity	Application progress
[0,5)	P1	5 ms
[5,6)	Context save/restore	None
[6,8)	P2	2 ms

There is no idle time: busy-time utilization is 100%, but useful-work fraction is 7/8=87.5%. Their numerators differ. The trace does not specify when P1's subsequent I/O or whole job completes.

Long-term admission selects jobs entering the system; short-term scheduling selects the next execution context; classical medium-term suspension/swapping changes memory pressure. These are different decision scales.

Fix the metric before comparing policies

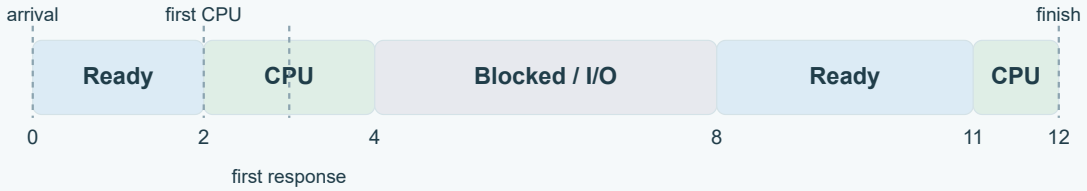
Let a be arrival, s first CPU dispatch, o first response production, and f completion.

Metric	Calculation	Distinction
Waiting W	Sum of ready intervals	Excludes CPU service and blocked time
Turnaround T	f−a	Includes intervening waits and I/O
First-CPU delay R_CPU	s−a	Does not imply application output
First-response delay R_out	o−a	Not complete output or job completion

Numerical scheduling exercises use R_CPU when they say response unless specified otherwise. The reference textbook § 6.2 describes interactive response production; that need not coincide with first dispatch. This is an explicit exercise convention, not an asserted identity between the events.

Why four milliseconds of I/O do not count as ready waiting

One task A: account for ready, execution and I/O intervals separately



Ready waiting	CPU service	Blocked I/O	Turnaround
2+3 = 5 ms	**2+1 = 3 ms**	**4 ms**	**12 ms**

First-CPU delay =2; first-response delay =3; 5+3+4 =12.

A arrives at 0: ready[0,2), CPU[2,4), blocked[4,8), ready[8,11), CPU[11,12). It first responds at 3. Thus W=2+3=5, service=3, blocked=4 and T=12: **5+3+4=12**.

R_CPU=2 and R_out=3. First execution, first response and completion differ. Counting I/O would incorrectly give W=9; T minus CPU service equals waiting only when other interval types are absent.

Define the selection rule, not just its name

General-policy examples use one CPU, one known CPU burst per job and zero switching cost. Unless overridden, ties follow arrival order then identifier. Intervals are half-open [start, end).

Policy	Selection key	Reconsideration
FCFS	Ready-enqueue order	Current releases CPU
Nonpreemptive SJF	Next CPU-burst length	Current releases CPU
SRTF	Remaining CPU demand	Arrivals/completions; shorter jobs may preempt
RR	FIFO and remaining quantum	Completion, blocking or quantum expiry
Priority	Explicitly ordered priority	Preemption depends on policy

Exact future burst lengths are a teaching assumption. Estimated lengths do not inherit guarantees that require exact lengths.

FCFS: short jobs can queue behind a long one

P1/P2/P3 all arrive at 0, require 8/4/2 ms and enqueue in that order.

Job	Execution	Completion f	Waiting W	Turnaround T
P1	[0,8)	8	0	8
P2	[8,12)	12	8	12
P3	[12,14)	14	12	14

Mean waiting is $20/3 \approx 6.67$ ms; mean turnaround is $34/3 \approx 11.33$ ms. Short jobs waiting together behind a long burst illustrate the convoy effect; interleaved I/O can further affect CPU/device coordination.

This does not make FCFS universally worst. Finite predecessors with finite service give a finite FIFO wait, but no uniform numerical bound follows without workload bounds.

SJF: reorder the same available batch

The 8/4/2 ms batch executes P3→P2→P1. **Rows remain in P1/P2/P3 order**, distinct from execution order.

Job	Execution	Completion f	Waiting W	Turnaround T
P1	[6,14)	14	6	14
P2	[2,6)	6	2	6
P3	[0,2)	2	0	2

Mean waiting is 8/3≈2.67 ms,4 ms lower than FCFS; P1 nevertheless finishes at 14 instead of 8. An improved average is not an improvement for every job.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The exchange argument and its assumptions

All jobs are available together, with one known burst each, nonpreemptive execution and zero switching cost. Adjacent jobs have lengths x>y and a shared preceding delay h.

Order	First wait	Second wait	Combined wait
x then y	h	h+x	2h+x
y then x	h	h+y	2h+y

Exchanging them reduces the sum by x−y>0. Their total execution is unchanged, so later start times are unchanged. Removing every inversion yields ascending lengths and minimum total/mean waiting. Fixed total service also gives minimum mean turnaround.

Different arrivals, deliberate idling, weights, deadlines or prediction error require their own argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Predicting the next burst

Exponential averaging uses $\hat{b}_{n+1} = \alpha b_n + (1-\alpha)\hat{b}_n$, where b_n is the latest measured burst, the hat denotes prediction and $0 \le \alpha \le 1$.

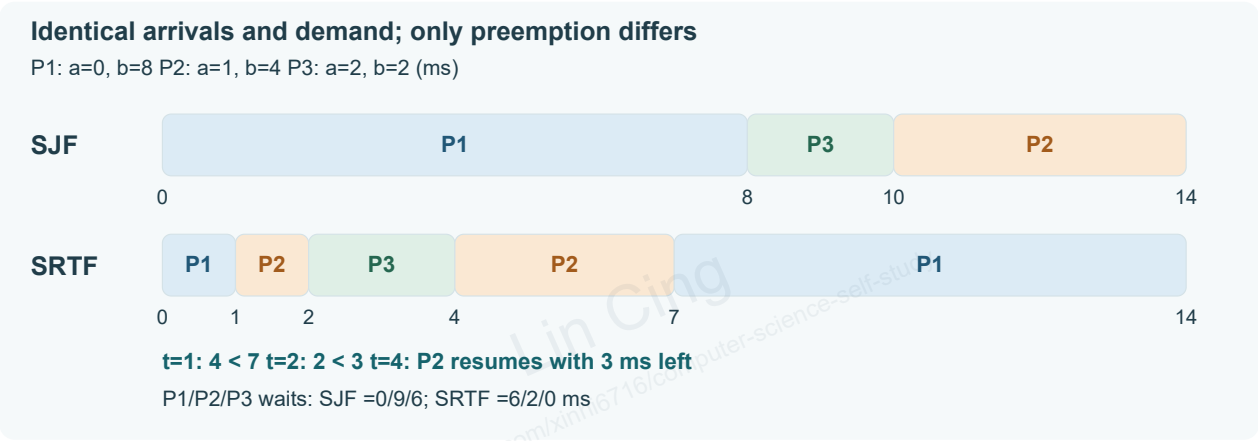
α=1/2; initial prediction 10 ms	Measured b_n	Next prediction
First	6	(6+10)/2=8
Second	4	(4+8)/2=6
Third	8	(8+6)/2=7

Expansion assigns older observations weights $\alpha(1-\alpha)^j$. For $0<\alpha<1$ these decay; $\alpha=0$ ignores new observations and $\alpha=1$ uses only the latest burst.

A phase change can defeat this predictor. Prediction does not remove the need for fairness or turn an estimated-shortest choice into an exact-shortest guarantee.

SRTF compares an arrival with remaining demand

Let P1=(arrival0, demand 8), P2=(1,4), P3=(2,2), in ms. Equal remaining demand retains current; otherwise choose the shortest.



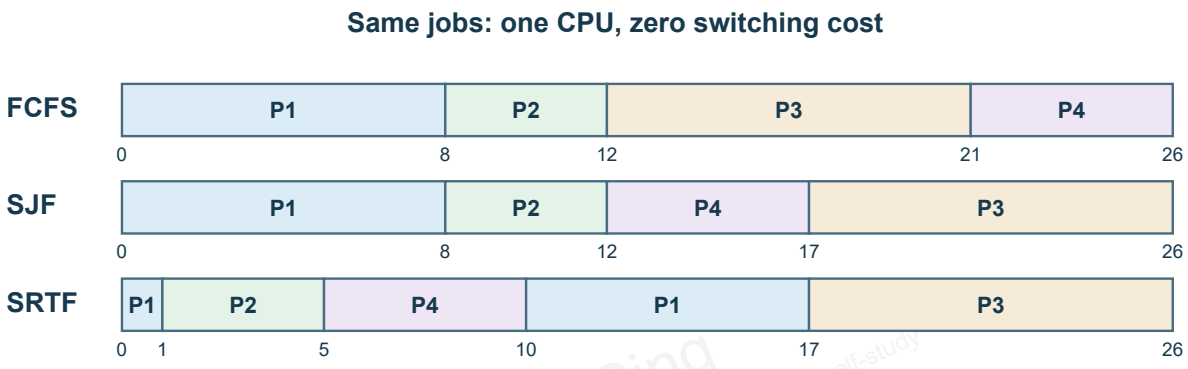
At 1, P1 has 7 left versus P2's4, so preempt. At 2, P2 has 3 left versus P3's2, so preempt again. At 4 choose P2's remaining 3 before P1's7. **P2 does not restart its original 4 ms.**

Reconstruct every SRTF event and metric

Event	Candidate remaining demand	Next segment	Reason
0	P1=8	P1[0,1)	Only candidate
1	P1=7, P2=4	P2[1,2)	4<7
2	P1=7, P2=3, P3=2	P3[2,4)	2 is smallest
4	P1=7, P2=3	P2[4,7)	3<7
7	P1=7	P1[7,14)	Only candidate

In P1/P2/P3 order: completion 14/7/4, turnaround 14/6/2, waiting 6/2/0 and mean waiting 8/3 ms. First-CPU delays are all 0. Nonpreemptive SJF instead completes 8/14/10 with waits 0/9/6 and mean 5 ms. Identical arrivals/demands make the policy comparison meaningful.

Three policies on the same arrivals and work



Assume one CPU, no switching cost, and one CPU burst per job. Inputs are P1=(arrival 0, demand 8), P2=(1,4), P3=(2,9), and P4=(3,5), in milliseconds. FCFS uses arrival order; nonpreemptive SJF picks the shortest available job whenever the CPU becomes free; SRTF picks the smallest remaining demand.

At t=1, SRTF preempts P1 because P2 needs 4 while P1 has 7 left. When P2 completes at t=5, P4's 5 is less than P1's 7 and P3's 9, so P4 runs next. SJF does not preempt on arrival, so P1 continues to t=8.

Derive waiting time from completion

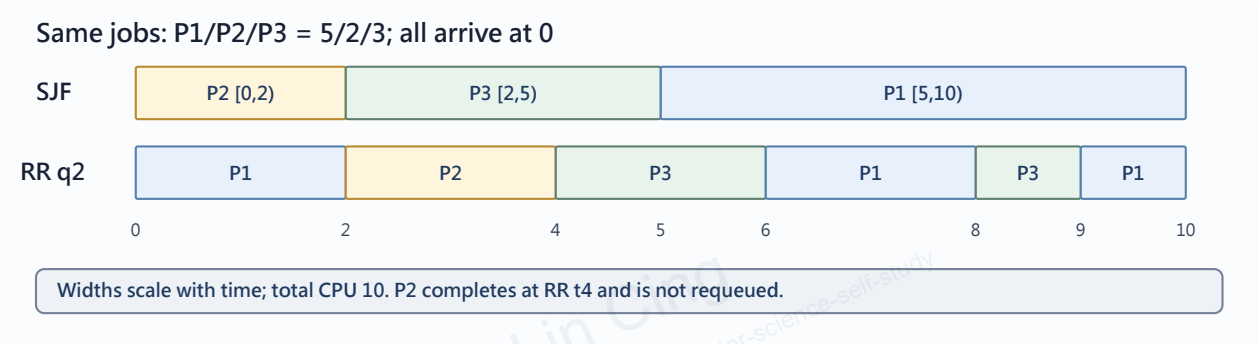
There is no I/O in this example, so each waiting time W is completion minus arrival minus CPU demand. Rows follow job IDs rather than execution order.

Job	FCFS completion / wait	SJF completion / wait	SRTF completion / wait
P1	8 / 0	8 / 0	17 / 9
P2	12 / 7	12 / 7	5 / 0
P3	21 / 10	26 / 15	26 / 15
P4	26 / 18	17 / 9	10 / 2
Mean wait	35/4 = 8.75	31/4 = 7.75	26/4 = 6.5

For example, under SRTF P1 runs for 1 ms in [0,1), leaving 7 ms. It resumes at t=10 and completes at t=17, so its waiting time is 17−0−8=9 ms. These results describe the stated workload; switching costs or a different objective can change the comparison.

RR: requeue only unfinished jobs

All arrive at 0; P1/P2/P3 demand 5/2/3 ms. Initial order P1→P2→P3, q=2, cost 0. Completed jobs leave; unfinished jobs rejoin the tail after a full slice.



In P1/P2/P3 order, RR completion is 10/4/9, waiting 5/2/6 and first-CPU delay0/2/4. P2 completes at 4 and cannot requeue. Mean waiting is 7/3 for SJF versus 13/3 ms for RR; both execute10 ms of work.

RR: separate a rotation bound from switching cost

For n continuously ready jobs, quantum q and handoff cost c, a job just requeued can face at most n−1 other quanta. Including the final handoff back to it gives bound **(n−1)q+nc**. This assumes a fixed eligible set, no higher scheduling class and no blocking.

Other jobs remain ready; c=0.1 ms	q	Useful fraction q/(q+c)
Larger quantum	4 ms	4/4.1≈97.56%
Smaller quantum	0.1 ms	0.1/0.2=50%

If q covers every entire burst, RR reduces to FCFS for that batch. A smaller q shortens rotations but raises overhead. With only one runnable job, expiry need not cause an actual cross-task switch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Priority, aging and long finite waits

Specify whether higher numbers mean higher priority. With strict priority, a low-priority job can starve if higher-priority jobs remain ready forever.

A finite trace shows only a finite wait. An infinite counterexample has L ready at 0 and high-priority H_k arriving at every integer k, each demanding 1 ms. Higher-priority service fills the CPU, so L never advances.

Aging increases priority with waiting. A finite-wait proof also needs conditions such as capped levels, fair equal-priority service and finite predecessors/service demands. An unspecified increase alone gives no bound. Changing real-time priorities requires renewed deadline analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed multilevel queues versus feedback queues

Multilevel queues keep jobs in fixed classes, with both intra-queue and inter-queue policies. Multilevel feedback queues (MLFQ) let jobs move between levels.

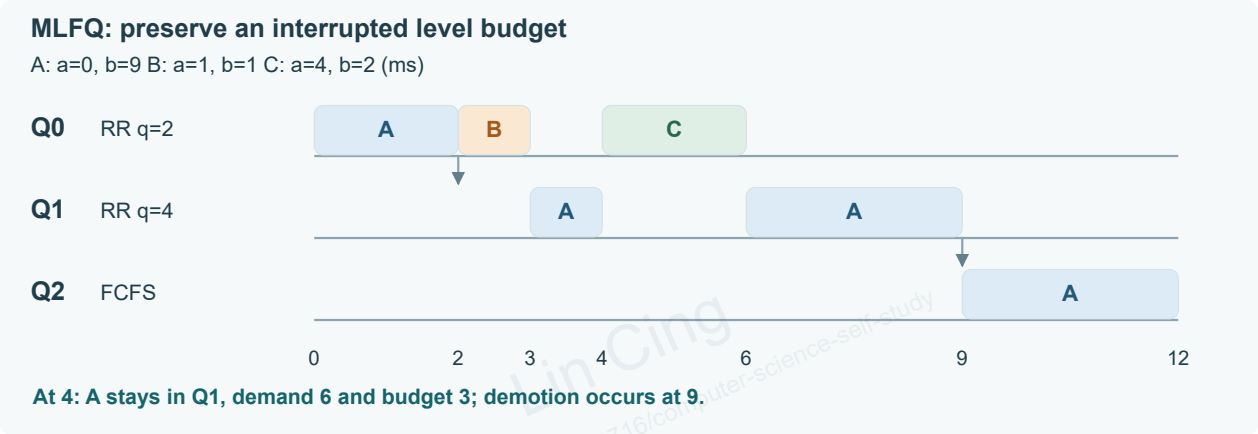
The following is one explicit teaching variant, **not a universal MLFQ definition**.

Rule	Chosen policy
Highest to lowest	Q0:RR q=2; Q1:RR q=4; Q2:FCFS
New arrival	Enter Q0; a nonempty higher queue preempts lower levels
Budget exhausted	Unfinished job drops one level; Q2 remains Q2
Higher-level preemption	Retain original-level head position and remaining budget
Promotion/blocking	No blocking or promotion in this finite example

Different requeue, boost or budget-reset rules can produce different schedules.

MLFQ: preemption does not replenish the budget

A=(arrival0, demand 9), B=(1,1), C=(4,2), all in ms, with the preceding rules.



At 4, A retains **Q1 budget 3 but CPU demand 6**. It resumes over 6–9, exhausts that budget, then completes in Q2 over 9–12. The preceding rules and following event table account for every segment.

Check every MLFQ queue and budget transition

States follow event processing and precede dispatch. Parentheses show remaining CPU/remaining level budget; Q2 has no quantum.

t	Q0	Q1	Q2	Next execution
0	A(9/2)	Empty	Empty	A0–2
2	B(1/2)	A(7/4)	Empty	B2–3
3	Empty	A(7/4)	Empty	A3–4
4	C(2/2)	A(6/3)	Empty	C4–6
6	Empty	A(6/3)	Empty	A6–9
9	Empty	Empty	A(3/—)	A9–12

Resetting A to a full 4 ms after preemption defines another policy and delays its demotion. Without aging/boosts, strict inter-level priority does not prevent starvation under infinite higher-level arrivals.

User-thread scheduling and CPU scheduling

A library manages user threads; the kernel selects kernel-visible execution entities. Process-contention scope (PCS) distributes available entities within a process; system-contention scope (SCS) competes for CPUs system-wide.

Mapping	Can T2 proceed while T1 blocks?	Can both execute on separate CPUs?
Many-to-one	Not if T1 blocks the sole kernel entity	No, only one kernel entity
One-to-one	If T2's own entity remains ready	Only with at least two available CPUs

If T2 needs 4 ms, one-to-one alone does not prove completion at 4: it must also run continuously from 0. Mapping makes execution possible; policy and load determine its timing.

Multicore balancing pays for locality changes

Processor affinity preserves useful cache/TLB state; load balancing moves work toward idle CPUs. Queue length alone cannot resolve the tradeoff.

One job must not be claimed by two CPUs. Synchronized migration removes its old queue entry, transfers unique ownership and enqueues it at the destination. A running job and an unstarted queued job are not interchangeable forms of stealing.

The following eight-core example fixes all jobs ready at 0, indivisible execution, no job on two CPUs at once and 1 ms per transfer plus warmup. These are model costs, not measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eight cores: account for transfer and warmup

Fix eight cores, all jobs ready at t=0, indivisible jobs, and no job running on two cores at once. A/B each need 8 ms, C/D each need 4 ms, and E/F/G/H/I/J each need 2 ms: 36 ms of useful work. CPU 0 initially queues [A, I, J]; CPU 1 through CPU 7 initially hold B, C, D, E, F, G, H respectively.

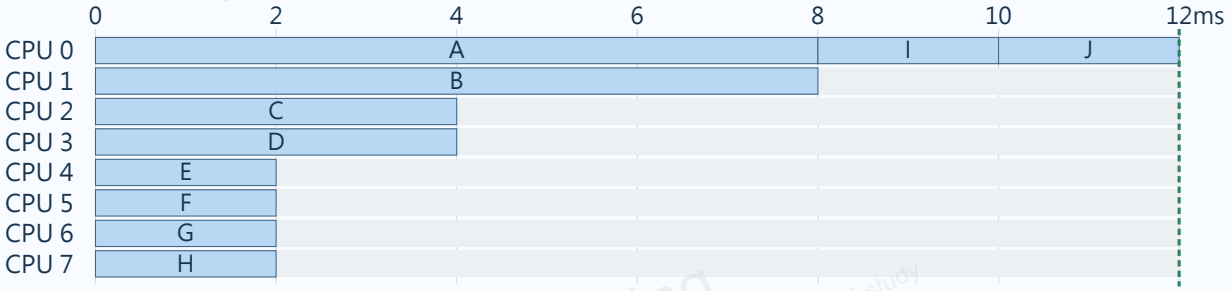
Without migration CPU 0 runs A[0,8), I[8,10), J[10,12), so makespan is 12 ms. At t=2 CPU 4/5 finish E/F. Under queue synchronization they each remove one still-queued job I/J and transfer its unique ownership. Charge each migration 0.5 ms for queue/transfer and 0.5 ms for cache warmup. Both destination cores pay overhead [2,3), then run I/J [3,5). CPU 0 finishes A at 8; makespan becomes 8 ms, speedup=12/8=1.5. Useful work remains 36 ms; added CPU work is 2 ms. These are specified model costs, not hardware measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Before migration: locate waiting work

No migration: CPU 0 runs A, I, J; makespan = 12 ms

Blue = useful work; gray = idle. All jobs ready at t=0. Identical 0–12 ms axes.



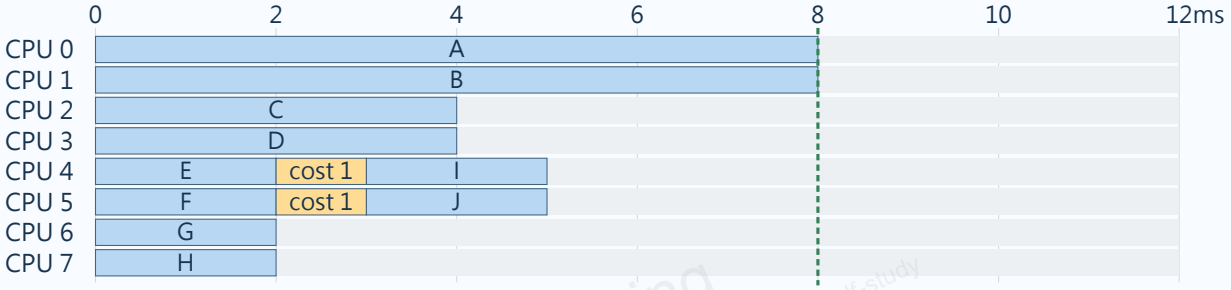
At t=2 CPU 4/5 finish E/F, but I/J still wait in CPU 0 queue behind A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

After migration: include its cost

Steal I/J at t=2: makespan including cost = 8 ms

Orange = 0.5 ms transfer + 0.5 ms cache warmup per job; blue = useful; gray = idle.



CPU 4/5: cost [2,3), I/J [3,5). A/B finish at 8. Useful work 36 ms + cost 2 ms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Job A takes 8 ms, so $T \geq 8$; this schedule attains that bound. With migration cost c, $T=\max(8,4+c)$, improving on 12 only if $c<8$. Affinity and dependencies may change the result.

Real-time scheduling: distinguish tasks, jobs, and deadlines

A periodic task τ_i releases successive jobs. Let C_i be its execution-time upper bound, T_i its period, and D_i its relative deadline. Unless stated otherwise, worked timelines use actual demand C_i for each job. With synchronous release at zero and job index k starting at zero,

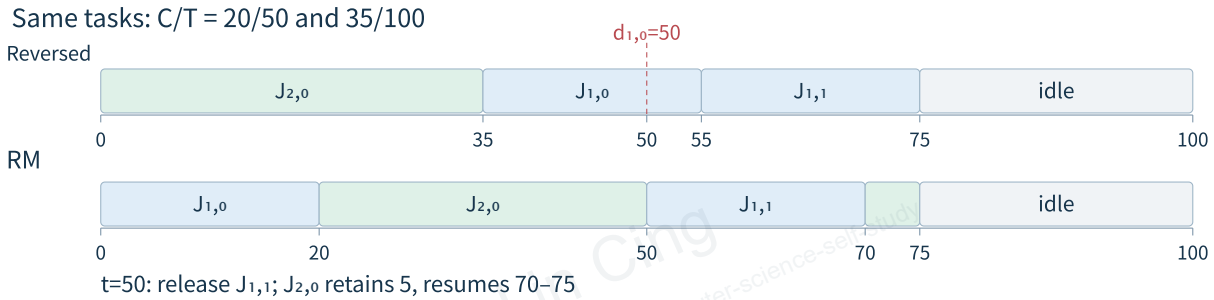
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

Base model: one CPU, immediate preemption, independent tasks, no lock blocking, I/O, self-suspension, switching or interrupt costs; $D_i=T_i$ except on the final page. A release adds a job to the ready set without necessarily changing the current job. Completion exactly at a deadline succeeds. At an event time t, account for execution up to t before processing new releases.

Quantity	Question answered	Distinction
C_i	How much CPU can each job require?	Not response time
T_i	How far apart are successive releases?	Not remaining time to deadline
D_i	How long after release may completion occur?	Absolute deadline is $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	What is the long-run CPU demand fraction?	Does not describe every short interval

RM gives shorter-period tasks higher fixed priority

Rate-monotonic (RM) assigns **higher fixed priority to a shorter period**, with preemption on a higher-priority release. Set τ_1=(C_1=20, T_1=D_1=50) and τ_2=(35,100,100); all times are milliseconds.



Reversed priorities delay $J_{1,0}$ to [35,55), missing50. Under RM, $J_{2,0}$ executes 30 in [20,50), is preempted, then completes its **remaining five units** in [70,75).

RM completions 20,70,75 meet deadlines 50,100,100. At 100 no old job remains and synchronous releases recur, proving repetition every 100 ms.

The RM utilization bound is sufficient, not a rejection rule

For n independent preemptible tasks with $D_i=T_i$ on the ideal single CPU, the classical **sufficient condition** is

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

For n=2 the bound is $2(\sqrt{2}-1)\approx 0.8284$. The preceding U=0.75 passes. The limit as n grows is $\ln 2 \approx 0.6931$. These are workload-independent guarantee bounds, **not a utilization ceiling for every RM workload**.

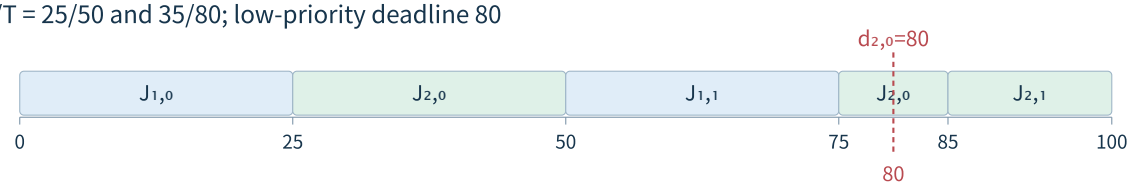
Counterexample: τ_1=(C=1, T=D=2), τ_2=(2,4,4) gives U=1. RM executes $J_{1,0}$ in [0,1), $J_{2,0}$ in [1,2), $J_{1,1}$ in [2,3), then the final unit of $J_{2,0}$ in [3,4). Completion times 1,3,4 meet deadlines 2,4,4. At t=4 the empty carry-in state and synchronous releases recur, so this schedule repeats.

Check assumptions, then apply the sufficient test. If it fails, use tighter interference analysis or a schedule whose recurring state is proved. U>0.8284 alone does not prove failure, and U≤1 does not certify an arbitrary fixed-priority policy.

RM can miss a deadline even below full utilization

Now set τ_1=(25,50,50), τ_2=(35,80,80). Utilization is 15/16=0.9375. RM still gives τ_1 higher priority.

RM: C/T = 25/50 and 35/80; low-priority deadline 80



At 80: old $J_{2,0}$ still needs 5; new $J_{2,1}$ is separate

$J_{2,0}$ executes 25 units in [25,50), leaving ten. $J_{1,1}$ preempts it in [50,75). It receives only five more units in [75,80), so **five remain at deadline 80**. The miss has already happened. If late jobs are retained, this job finishes at 85. The new $J_{2,1}$ released at 80 is a separate job.

Long-run spare capacity does not prevent fixed priorities from crowding a particular interval. The pending demand at 80 proves failure; exceeding a sufficient utilization bound does not.

Count interference to check the RM completion time

For two synchronous tasks, τ_1 higher priority, $C_1 \le T_1$ and no blocking, $[0, R)$ contains $\lceil R/T_1 \rceil$ higher-priority releases at $0, T_1, 2T_1, \ldots$. **A release exactly at R is excluded.**

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

For the synchronous sequence where each job actually needs its bound C_i, iterate from C_2 to the earliest fixed point; stop if $w > D_2$. This is a worst-case sequence, not a claim that every lower-demand run misses its deadline.

Monotonicity makes iterates nondecreasing. Any fixed point $R^* \le D_2$ bounds every iterate by induction; an iterate $> D_2$ excludes it. Only the first synchronous job is covered; other phases, tasks or blocking need further analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three workload sets: evaluate each iteration

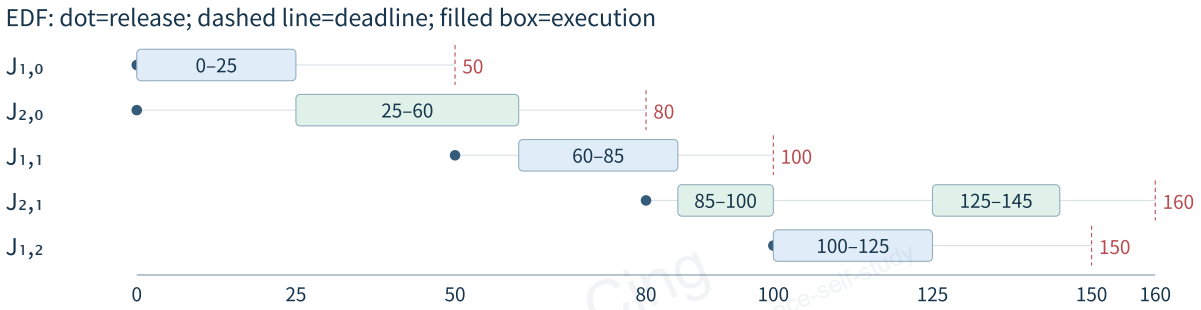
Task pairs (C, T)	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	Check
(20,50),(35,100)	35	55	75	75	$75 \le 100$
(25,50),(35,80)	35	60	85	85	$85 > 80$
(1,2),(2,4)	2	3	4	4	Equality succeeds

For row 2: $35+\text{ceil}(35/50)\times 25=60$, then $35+\text{ceil}(60/50)\times 25=85>80$, beyond the deadline. Row 1 stabilizes at $75 \le 100$. The preceding interference argument gives these iterations their meaning.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EDF compares absolute deadlines of individual jobs

EDF selects the **earliest absolute deadline** among ready jobs. Use $\tau_1=(25,50,50)$,$\tau_2=(35,80,80)$; ties retain current, otherwise task/job order.



At 50, current $J_{2,0}$'s80 beats new $J_{1,1}$'s100: **continue to 60**. At 100, new $J_{1,2}$'s150 beats current $J_{2,1}$'s160: preempt.

The first 150 ms alone cannot certify every future job; the theorem below does. Preemption preserves remaining execution.

Verify each EDF choice against the ready set

Each row is after processing that event and before executing the next interval. Parentheses contain (remaining CPU, absolute deadline); completed jobs are absent.

t (ms)	Ready jobs	Next execution	Reason
0	$J_{1,0}$(25,50), $J_{2,0}$(35,80)	$J_{1,0}$:0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$:25–50	Only ready job
50	$J_{1,1}$(25,100), $J_{2,0}$(10,80)	$J_{2,0}$:50–60	80<100; continue
60	$J_{1,1}$(25,100)	$J_{1,1}$:60–80	Only ready job
80	$J_{1,1}$(5,100), $J_{2,1}$(35,160)	$J_{1,1}$:80–85	100<160; continue

At both 50 and 80 a new release changes the ready set while current remains earliest-due. Release alone does not imply preemption.

The same EDF rule does preempt at 100

Continue with (remaining CPU, absolute deadline) for ready jobs.

t (ms)	Ready jobs	Next execution	Reason
85	$J_{2,1}$(35,160)	$J_{2,1}$:85–100	Only ready job
100	$J_{1,2}$(25,150), $J_{2,1}$(20,160)	$J_{1,2}$:100–125	150<160; preempt
125	$J_{2,1}$(20,160)	$J_{2,1}$:125–145	Only ready job
145	Empty	Idle:145–150	No ready job

The first three τ_1 completions are 25,85,125 against 50,100,150. The first two τ_2 completions are 60,145 against 80,160. **Release, priority reevaluation and switching are distinct events.** Releases at 50 and 80 do not change current.

Why does ideal EDF need only U≤1 when D=T?

Use the stated one-CPU model, starting without carry-in at t=0; tasks may have different nonnegative release phases. If U>1, consider the legal sequence in which every job actually needs its upper bound C_i. Demand due by time t is Ut−O(1), eventually exceeding available CPU time. No policy can guarantee all legal demand sequences meet every deadline; an actual run with lower demands may still succeed.

For sufficiency suppose U≤1, yet EDF first misses a deadline b. Consider only jobs with deadlines at most b. Let [a, b) be their last continuously backlogged interval, of length L=b−a. None of these jobs was pending immediately before a, so the relevant jobs all release at or after a. Throughout the interval EDF executes only such jobs.

Since $D_i=T_i$, at most $\lfloor L/T_i \rfloor$ jobs of task i can both release and become due in this interval: the first cannot be due before $a+T_i$ and later deadlines are separated by T_i. Thus their total demand satisfies

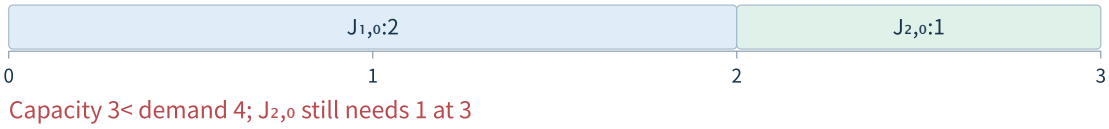
$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor \, C_i \le L \sum_i C_i/T_i = LU \le L.$$

A miss after L units of uninterrupted service instead requires demand **greater than L**, a contradiction. Hence U≤1 is sufficient and, with the necessity argument, necessary and sufficient to guarantee deadlines for all legal demand sequences in this model. The proof bounds every relevant interval rather than relying on an average alone.

Shorter relative deadlines can defeat low utilization

Let both tasks have C=2, T=10, D=3, released together at 0. U=0.4, yet four CPU units must complete by 3. A single CPU supplies only three in [0,3). **Every policy, including EDF, must miss at least one deadline.**

C=2, T=10, D=3: both release 0; four units required by 3



Capacity 3< demand 4; $J_{2,0}$ still needs 1 at 3

For $D_i \leq T_i$ the maximum demand both released and due within length L is

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

The first job needs D_i time; each additional T_i permits another. At L=3 total demand 4>3 proves impossibility. Passing just one length cannot certify all intervals. Priority inheritance handles specific lock-induced inversion; it cannot fix this capacity contradiction.

A system name is not a scheduling rule

Fix the following historical versions and event models: Solaris TS/IA, Windows 7 and Linux 3.2 CFS. These are mechanism comparisons, not a modern-product benchmark.

Case	Selection mechanism	Required conditions
Solaris TS/IA	Class priority mapped to global priority; expiry and wakeup change levels	Dispatch table, quanta and return rules
Windows 7	32 ready priorities, equal-level rotation and dynamic boosts	Base level, boosts and tick units
Linux 3.2 CFS	Vruntime tree with fairness-bounded candidate hints	Weights, timer and wakeup thresholds

Do not reduce all three to a universal RR quantum. The following CFS case fixes its event model before deriving the timeline.

A fixed Linux 3.2 event model

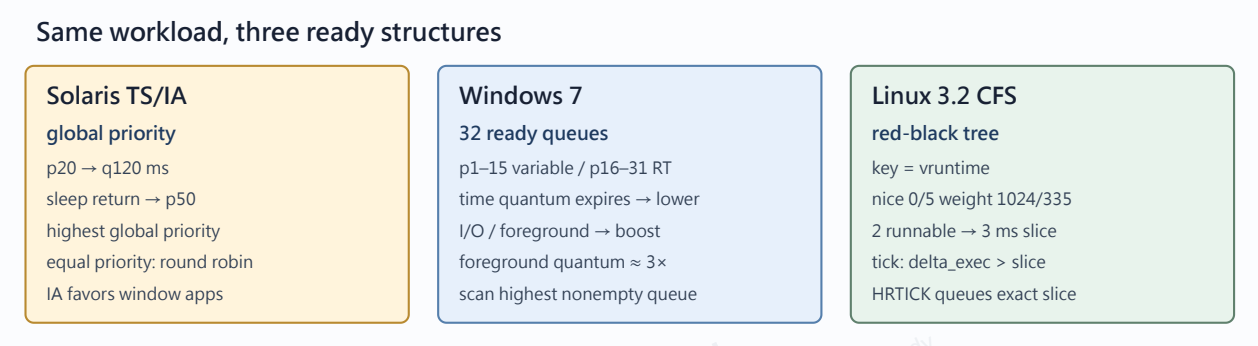
One CPU: P1 and P2 are ready at 0 and require 8 and 4 ms; P3 requires 2 ms and wakes at 9. All weights are 1024, initial vruntime is normalized to 0, and P1 wins the initial tie. Ignore switching, interrupt and timer-jitter costs.

Set target latency=6 ms, minimum granularity=0.75 ms, wakeup granularity=1 ms and GENTLE_FAIR_SLEEPERS=true. Compile with CONFIG_SCHED_HRTICK=y; enable the runtime HRTICK feature, use an active high-resolution timer, and set DOUBLE_TICK=false. These assumptions do not imply HRTICK is enabled on every Linux 3.2 system.

For $n \leq 8$ runnable tasks, period=6 ms; otherwise period=0.75n ms. Equal weights give an ideal slice of period/n: 3 ms for two tasks and 2 ms for three.

At t=9, process P3's enqueue before the old HRTICK expiry handler. Ordinary ticks may occur, but do not cause additional switches in this model.

Three ready structures use different selection rules



Current is outside the ready tree. A sleeping entity is ineligible until its wakeup event; selection and sleep placement must be tracked separately.

CFS event trace

States are after event processing and selection. Ready excludes current; slice units are ms.

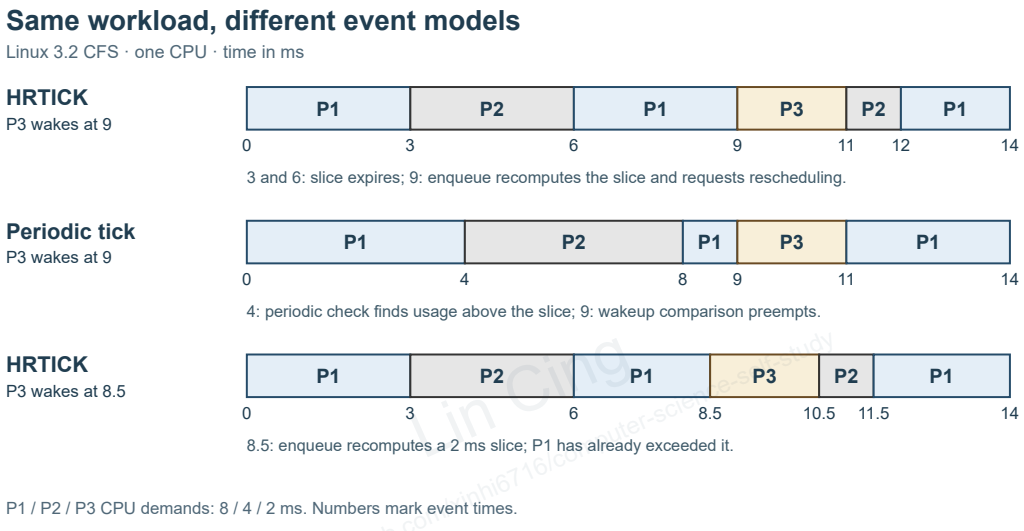
Time/event	Current/slice	Other ready	min_vruntime	Selection reason
0 initial	P1/3	P2@0	0	P1 inserted first on tie
3 HRTICK	P2/3	P1@3	0	P1's slice expires
6 HRTICK	P1/3	P2@3	3	Existing leftmost wins tie
9 P3 enqueue	P3/2	P2@3, P1@6	3	Recomputed slice is 2; P1 has run 3; enqueue update requests rescheduling
11 P3 complete	P2/3	P1@6	3	Ready key 3 precedes 6
12 P2 complete	P1/6	none	6	P1 alone, 2 ms remaining
14 P1 complete	none	none	8	No work remains

CFS event trace (continued)

With GENTLE_FAIR_SLEEPERS, P3 is placed at max(0,3−6/2)=0. Its key may be below min_vruntime, but the baseline does not move backward. The enqueue-triggered HRTICK update has already requested rescheduling; the 6−0>1 wakeup comparison is not the cause of the switch in this event order.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compare the timer-dependent timelines



Tick, HRTICK, and wakeup sensitivity

Settings	Timeline (ms)	Completion P1/P2/P3
HRTICK; wake at 9	P1 0–3, P2 3–6, P1 6–9, P3 9–11, P2 11–12, P1 12–14	14/12/11
No HRTICK; 1 ms tick; wake at 9	P1 0–4, P2 4–8, P1 8–9, P3 9–11, P1 11–14	14/8/11
HRTICK; wake at 8.5	P1 0–3, P2 3–6, P1 6–8.5, P3 8.5–10.5, P2 10.5–11.5, P1 11.5–14	14/11.5/10.5

The ordinary tick's slice comparison is strict: 3>3 is false, so P1 first leaves at t=4 without HRTICK. In the final row P3's enqueue reduces P1's ideal slice to 2 ms after P1 has run 2.5 ms; hrtick_update requests rescheduling. In the second row, the wake comparison 5−2>1 requests it instead. These are different triggering paths.

Tick checks, wakeup comparison and buddy hints

After minimum granularity, the ordinary tick may also compare the current task's vruntime with the leftmost candidate. Arrival or wakeup alone does not guarantee preemption.

Linux 3.2's wakeup-preemption path checks whether rescheduling has already been requested. If P3's enqueue has already triggered hrtick_update, the wakeup comparison is not the cause of that switch; if not, the wakeup rule must be evaluated separately.

Last/next/skip buddy are candidate hints for locality and scheduling overhead, constrained by fairness. They are not absolute priorities, and selecting P3 does not by itself prove that P3 was nominated next buddy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nice weights and historical comparisons

With only A nice 0 and B nice 5 runnable, total weight=1359 and period=6 ms.

Task	Weight	Ideal runtime	vruntime per real ms	CPU share
A	1024	$6\times1024/1359\approx4.521$ ms	1 ms	75.35%
B	335	$6\times335/1359\approx1.479$ ms	$1024/335\approx3.057$ ms	24.65%

Weights are not linear nice percentages. Above eight runnable tasks the period grows to preserve minimum granularity instead of shrinking slices without bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the same workload need not give one system timeline

System	Selection mechanisms	Inputs required to reconstruct a timeline
Solaris TS/IA	Class/global priorities, expiry and wakeup adjustments	Initial priorities, dispatch table, quanta and event ordering
Windows 7	Multiple ready priorities, equal-level rotation and dynamic boosts	Initial priorities, quantum, tick unit, boosts and queue order
Linux 3.2 CFS	Vruntime, weights, timer and sleep placement	Weights, timer configuration, wake time and tie rules

CPU demands and wake time alone do not determine a Solaris or Windows Gantt chart. Without the missing parameters, completion times cannot be derived. The Linux timeline above follows from its stated assumptions; it is not a performance ranking of the three systems.

Comparing policies requires comparable evidence

Method	What it can establish	What it cannot establish alone
Deterministic trace	Exact decisions and metrics for specified jobs	The same outcome under other arrivals
Queueing model	Means under arrival and service assumptions	Tail behavior outside those assumptions
Simulation	Behavior under modeled costs and distributions	Costs omitted from the model
Measurement	Observations on a fixed version and machine	All machines and future workloads

Use the same inputs and account for unfinished jobs. Reporting only completed short jobs biases a comparison that omits a long job still waiting.

Waiting, first response and throughput answer different questions

One CPU; all arrivals 0; P1/P2/P3 bursts5/2/3 ms; no I/O or switching cost. Response here means first CPU. Nearest-rank p99 for three observations is the batch maximum, not a population-tail estimate.

Policy	Completion P1/P2/P3	Waiting P1/P2/P3	Mean waiting	First response P1/P2/P3	Batch p99 response
FCFS, initial P1/P2/P3	5/7/10	0/5/7	4	0/5/7	7
SJF, P2/P3/P1	10/2/5	5/0/2	7/3	5/0/2	5
RR q=2, initial P1/P2/P3	10/4/9	5/2/6	13/3	0/2/4	4

Throughput is 3/10 jobs/ms for every policy. SJF minimizes this batch's mean wait; RR lowers its maximum first-CPU delay. Three observations do not estimate population p99. Random-workload claims would require distributions and independent replications.

A busy CPU can still leave jobs waiting longer

Keep RR q=2, all arrivals 0 and initial order P1/P2/P3. Multiply each burst by k. Run the queue head for min(2, remaining), then append it only if unfinished. Ignore switching cost; units are ms.

k; bursts	Completion P1/P2/P3	Waiting P1/P2/P3	Throughput jobs/ms	Batch p99 first-CPU response	Maximum waiting
1; 5/2/3	10/4/9	5/2/6	3/10	4	6
2; 10/4/6	20/10/16	10/6/10	3/20	4	10
3; 15/6/9	30/16/25	15/10/16	1/10	4	16

All three batches first run at 0/2/4 and keep the CPU 100% busy. Maximum waiting still grows6→10→16. Fast first dispatch does not imply fast completion. A finite-batch comparison does not prove starvation freedom under continuing arrivals.

Replay every slice for k=2

Completion 20/10/16 minus demand 10/4/6 gives waiting 10/6/10; no I/O. Each cell follows chronological order.

Time	Jobs	Remaining	Outcome
0–2; 2–4	P1; P2	8; 2	Both requeue
4–6; 6–8	P3; P1	4; 6	Both requeue
8–10; 10–12	P2; P3	0; 2	P2 completes; P3 requeues
12–14; 14–16	P1; P3	4; 0	P1 requeues; P3 completes
16–18; 18–20	P1; P1	2; 0	Same job continues to completion

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study